

# Entity Authentication

MODULE V

# Introduction to Entity Authentication

- Entity authentication is a technique designed to let one party prove the identity of another party.
- An entity can be a person, a process, a client, or a server.
- The entity whose Identity needs to be proved is called as *claimant*.
- The party that tries to prove the identity of the claimant is called the *verifier*.
- When Bob tries to prove the identity of Alice, Alice is the claimant, and Bob is the verifier.

# Data Origin versus Entity origin

- There are two differences between message authentication (data origin authentication) and entity authentication.
- Message authentication might not happen in Realtime; entity authentication does. In the former, Alice sends a message to Bob. When Bob authenticates the message, Alice may or may not be present in the communication process. On the other hand, when Alice requests entity authentication, there is no real message communication involved until Alice is authenticated by Bob. Alice needs to be online and to take part in the process. Only after she is authenticated can messages be communicated between Alice and Bob.
- Data origin authentication is required when an email is sent from the Alice to Bob. Entity authentication is required when Alice gets cash from an automatic teller machine.
- Second Important point, message authentication simply authenticates one message; the process needs to be repeated for each new message. Entity authentication authenticates the claimant for the entire duration of a session.

# Verification categories

In entity authentication, the claimant must identify herself to the verifier. This can be done with one of the three kinds of witnesses: something known, something possessed, or something inherent.

**Something known:** This is a secret known only by the claimant that can be checked by the verifier. Examples are a password, a PIN, a secret key, and a private key.

**Something possessed:** This is something that can prove the claimant's identity. Examples are a passport, a driver's license, an identification card, a credit card, and a smart card.

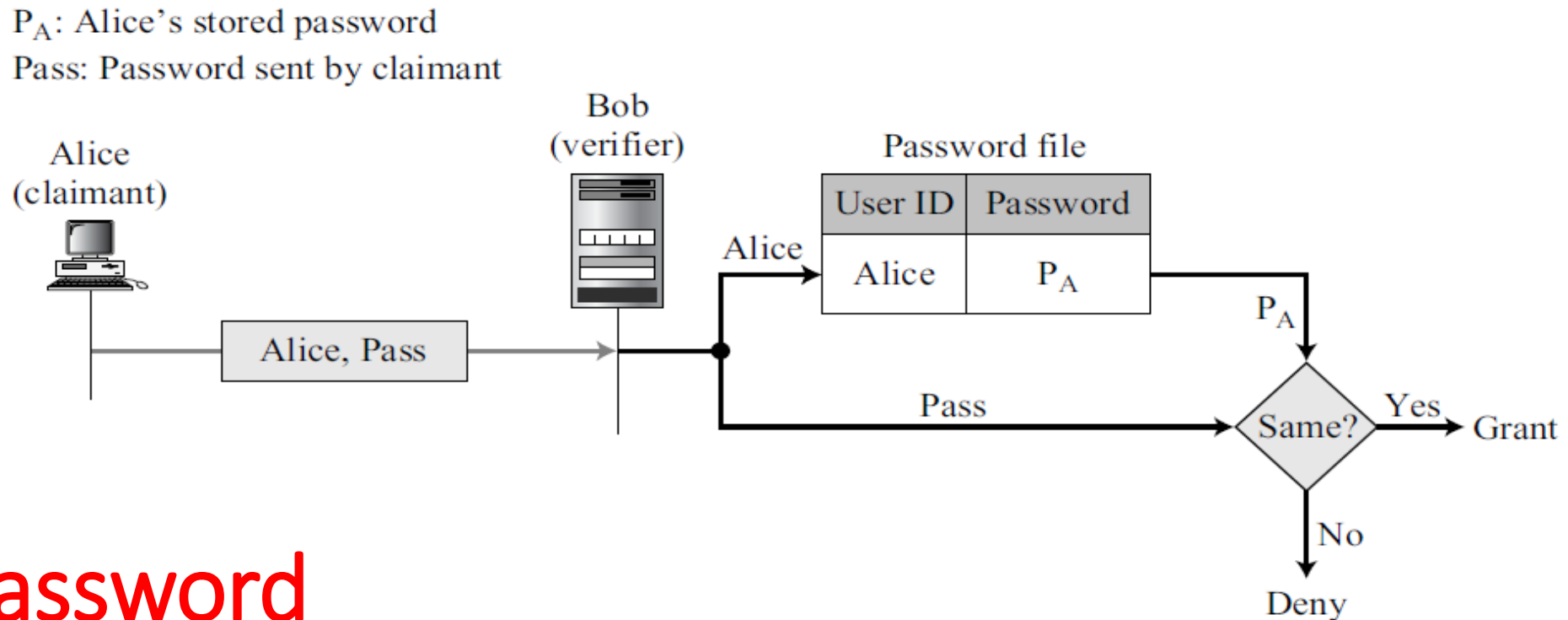
**Something Inherent:** This is an inherent characteristics of the claimant. Examples are conventional signatures, fingerprints, voice, facial characteristics, retinal pattern, and handwriting.

# Passwords

- To simplest and oldest method of entity authentication is the password-based authentication, where the password is something that the claimant knows. A password is used when a user needs to access a system to use the system's resources (login). Each user has a user identification that is public, and a password that is private. There are two types: **fixed password** and the **one-time password**.

- A fixed password is a password that is used over and over again for every access. Several schemes have been built, one upon the other.

- **First approach:** In the very rudimentary approach, the system keeps a table (a file) that is sorted by user identification. To access the system resources, the user sends her user identification and password, in plaintext, to the system. The system uses the identification to find the password in the table. If the password sent by the user matches the password in the table, access is granted; otherwise, it is denied



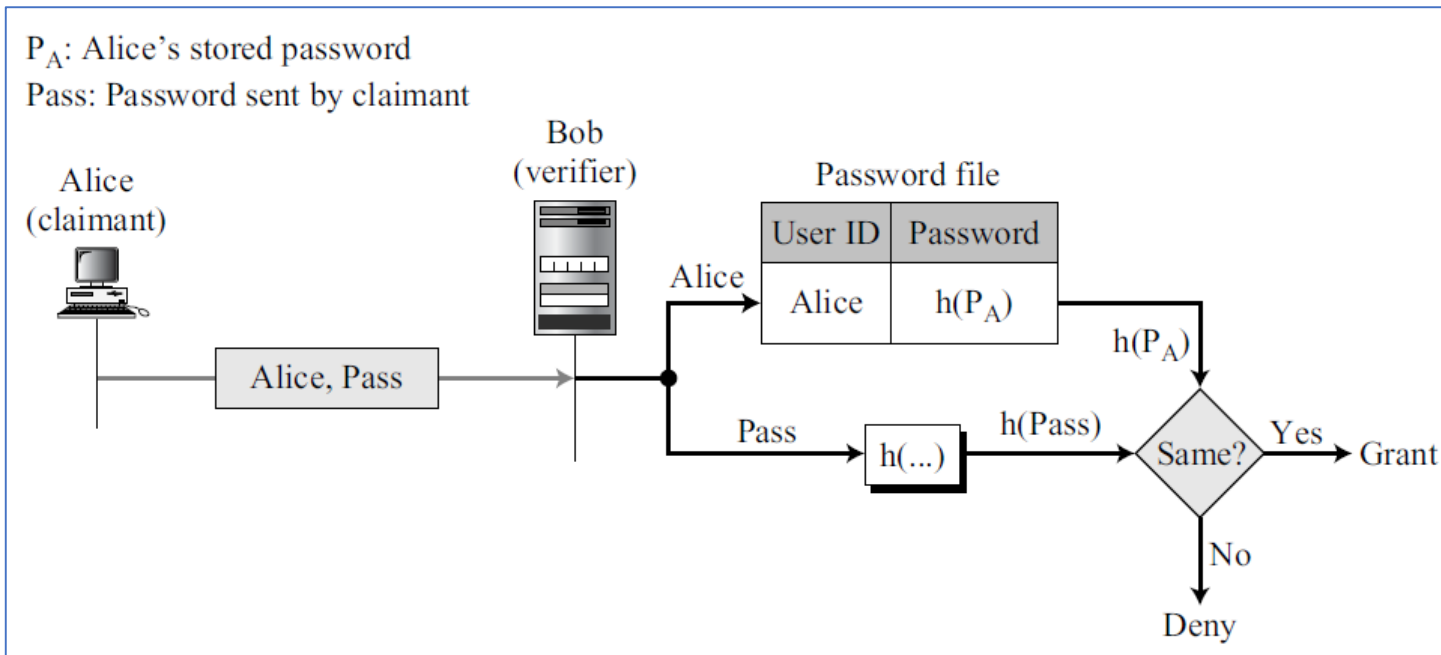
Fixed password

# Attacks on the first approach

- **Eavesdropping:** Eve can watch Alice when she types her password
- **Stealing a password:** The second type of attack occurs when Eve tries to physically steal Alice's password.
- **Accessing a password file:** Eve can hack into the system and get access to the ID/password file. Eve can read the file and find Alice's password or even change it.
- **Guessing:** Using a guessing attack, Eve can log into the system and try to guess Alice's password by trying different combinations of characters.

# Second approach

- A more secure approach is to store the hash of the password (instead of the plaintext password) in the password file.
- Any user can read the contents of the file, but, because the hash function is a one-way function, it is almost impossible to guess the value of the password.



When the user sends the ID and the password, the system creates a hash of the password and then compares the hash value with the one stored in the file. If there is a match, the user is granted access; otherwise, access is denied. In this case, the file does not need to be read protected.

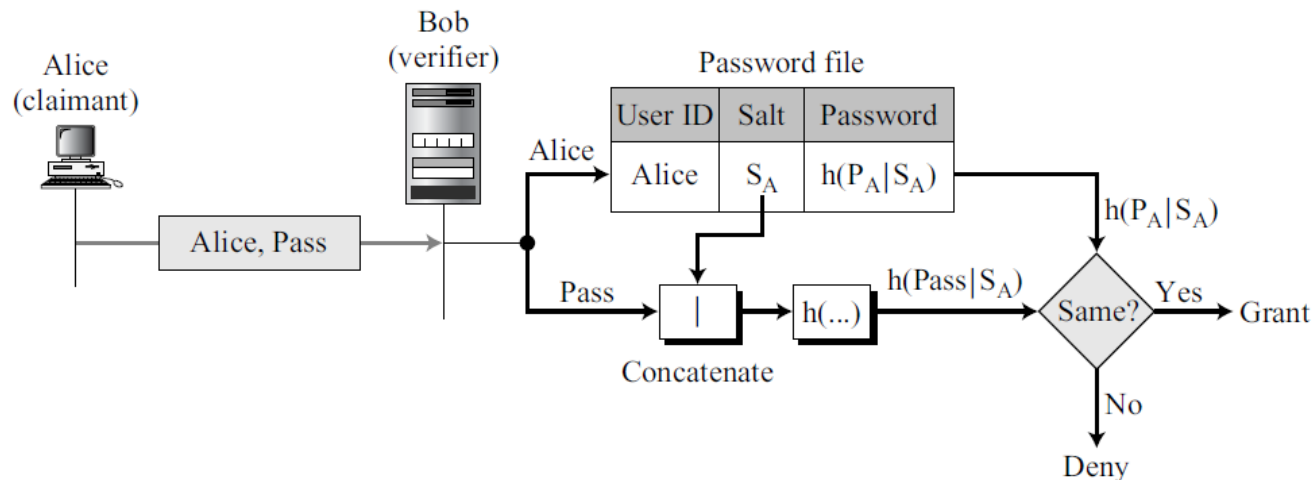
# Attack on second approach

- The hash function prevents Eve from gaining access to the system even though she has the password file. However, there is still the possibility of dictionary attack. In this attack, Eve is interested in finding one password, regardless of the user ID. For example, if the password is 6 digits, Eve can create a list of 6-digit numbers (000000 to 999999), and then apply the hash function to every number; the result is a list of one million hashes. She can then get the password file and search the second-column entries to find a match. This could be programmed and run offline on Eve's private computer. After a match is found, Eve can go online and use the password to access the system. The third approach shows how to make this attack more difficult.

# Third approach

- The third approach is called salting the password. When the password string is created, a random string, called the salt, is concatenated to the password. The salted password is then hashed. The ID, the salt, and the hash are then stored in the file. Now, when a user asks for access, the system extracts the salt, concatenates it with the received password, makes a hash out of the result, and compares it with the hash stored in the file. If there is a match, access is granted; otherwise, it is denied.

$P_A$ : Alice's password  
 $S_A$ : Alice's salt  
Pass: Password sent by claimant



Salting makes the dictionary attack more difficult. If the original password is 6 digits and the salt is 4 digits, then hashing is done over a 10-digit value. This means that Eve now needs to make a list of 10 million items and create a hash for each of them. The list of hashes has 10 million entries, and the comparison takes much longer. Salting is very effective if the salt is a very long random number.

## Fourth approach

- In the fourth approach, two identification techniques are combined. A good example of this type of authentication is the use of an ATM card with a PIN (personal identification number). The card belongs to the category “something possessed ” and the PIN belongs to the category “something known. The PIN is a password that enhances the security of the card. If the card is stolen, it cannot be used unless the PIN is known. The PIN number, however, is traditionally very short so it is easily remembered by the owner. This makes it vulnerable to the guessing type of attack.

# One Time Password

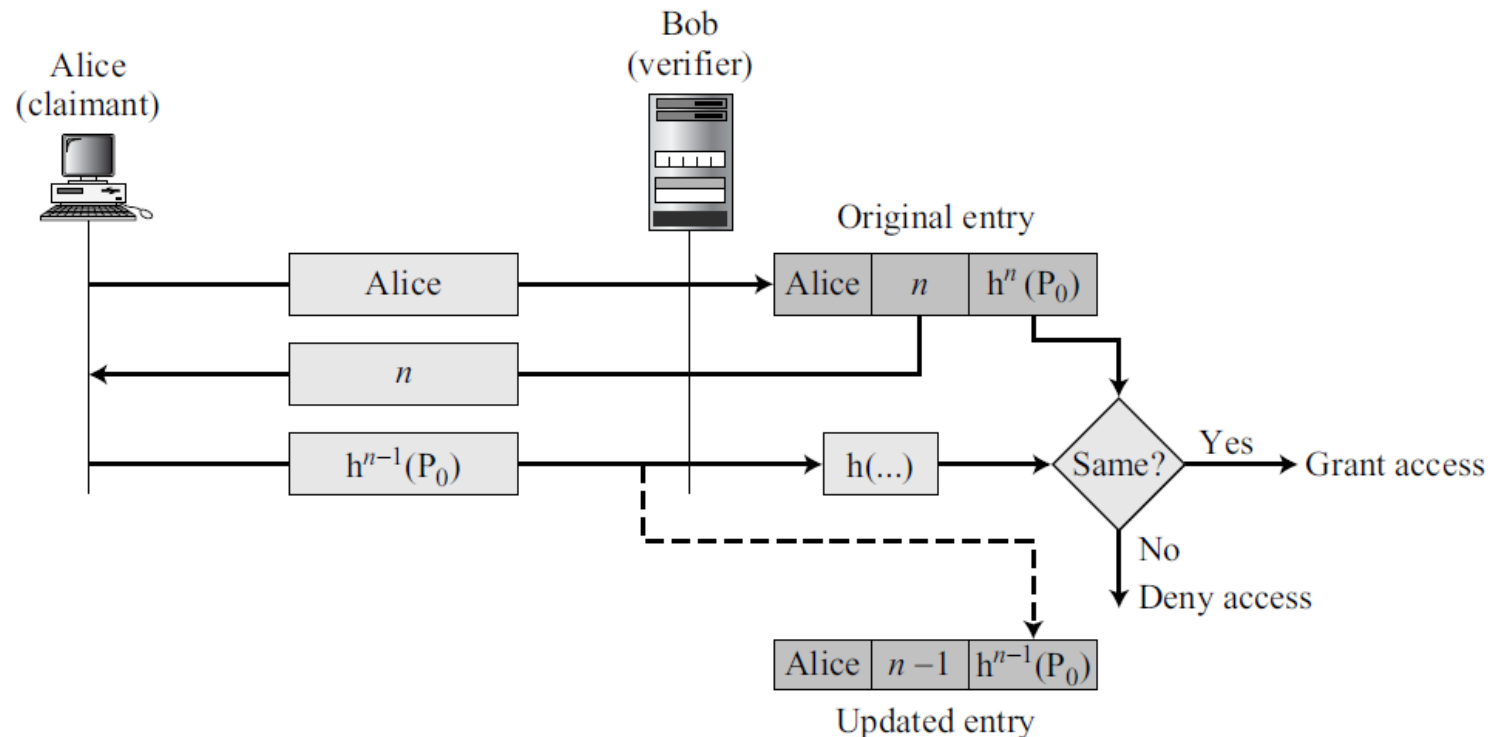
- A one-time password is a password that is used only once. This kind of password makes eavesdropping and salting useless .
- **First Approach.** In the first approach, the user and the system agree upon a list of passwords.
- Each password on the list can be used only once.
- There are some drawbacks to this approach.
  - First, the system and the user must keep a long list of passwords.
  - Second, if the user does not use the passwords in sequence, the system needs to perform a long search to find the match.
- This scheme makes eavesdropping and reuse of the password useless. The password is valid only once and cannot be used again.

# One Time Password

- **Second Approach.** In the second approach, the user and the system agree to sequentially update the password.
- The user and the system agree on an original password,  $P_1$ , which is valid only for the first access.
- During the first access, the user generates a new password,  $P_2$ , and encrypts this password with  $P_1$  as the key.  $P_2$  is the password for the second access.
- During the second access, the user generates a new password,  $P_3$ , and encrypts it with  $P_2$ ;  $P_3$  is used for the third access. In other words,  $P_i$  is used to create  $P_{i+1}$ .
- Of course, if Eve can guess the first password ( $P_1$ ), she can find all of the subsequent ones.

# One Time Password

- **Third Approach.** The user and the system create a sequentially updated password using a hash function. In this approach, elegantly devised by Leslie Lamport, the user and the system agree upon an original password,  $P_0$ , and a counter,  $n$ . The system calculates  $h^n(P_0)$ , where  $h^n$  means applying a hash function  $n$  times. In other words,
  - $h^n(x) = h(h^{n-1}(x))$   $h^{n-1}(x) = h(h^{n-2}(x)) \dots h^2(x) = h(h(x))$   $h^1(x) = h(x)$
  - The system stores the identity of Alice, the value of  $n$ , and the value of  $h^n(P_0)$ .



# One Time Password- third approach cntd..

- When the system receives the response of the user in the third message, it applies the hash function to the value received to see if it matches the value stored in the entry. If there is a match, access is granted; otherwise, it is denied. The system then decrements the value of  $n$  in the entry and replaces the old value of the password,  $h^n(P_0)$  with the new value  $h^{n-1}(P_0)$ .
- When the user tries to access the system for the second time, the value of the counter it receives is  $n - 1$ . The third message from the user is now  $h^{n-2}(P_0)$ . When the system receives this message, it applies the hash function to get  $h^{n-1}(P_0)$ , which can be compared with the updated entry.
- The value of  $n$  in the entry is decremented each time there is an access. When the value becomes 0, the user can no longer access the system; everything must be set up again. For this reason, the value of  $n$  is normally chosen as a large number such as 1000.